ancify: A Config-Driven Pipeline for Ancestral Allele Polarization Using Outgroup Alignments

Kevin Korfmann[1]

[1]Independent Researcher

February 19, 2026

Abstract

Determining the ancestral state of segregating alleles is a prerequisite for constructing unfolded site frequency spectra, detecting natural selection, and inferring demographic history. We present **ancify**, an open-source, species-agnostic Python pipeline that infers per-position ancestral alleles from pairwise net-AXT alignments with multiple outgroup species. The tool implements three complementary inference methods: (i) a two-tier inner/outer outgroup voting scheme that guards against incomplete lineage sorting, (ii) Fitch parsimony on a user-supplied phylogenetic tree, and (iii) a LightGBM gradient-boosted classifier that learns substitution biases from data. All methods produce FASTA output with case-encoded confidence levels. An optional GPU-accelerated backend using PyTorch reduces genome-wide processing time from approximately 12 hours to under 2 minutes for the human genome. Validation against the Ensembl EPO 13-primate ancestral reference yields >99.9% agreement on human autosomes. ancify is distributed under the MIT license and is available at `https://github.com/kevinkorfmann/ancify`.

Keywords: ancestral allele, polarization, site frequency spectrum, outgroup alignment, incomplete lineage sorting, Fitch parsimony, GPU acceleration, population genetics

1 Introduction

Understanding the direction of evolutionary change at polymorphic sites is fundamental to population genetics. The distinction between the *ancestral* allele—the state present in the most recent common ancestor of a sample—and the *derived* allele—the product of a more recent mutation—underlies virtually all inference about demographic history, natural selection, and mutational processes [Kimura, 1983, Watterson, 1975].

The *site frequency spectrum* (SFS), the distribution of allele frequencies across segregating sites in a population, is one of the most widely used summary statistics in population genetics. In its *unfolded* (polarized) form, the SFS distinguishes rare derived alleles from rare ancestral alleles, providing substantially more information than the symmetric *folded* SFS. Tools for demographic inference [Gutenkunst et al., 2009, Jouganous et al., 2017], selection scans [Nielsen et al., 2005, Fay and Wu, 2000], and mutation rate estimation [DeGiorgio et al., 2016] typically require or strongly benefit from the unfolded SFS.

Ancestral state determination relies on the parsimony principle: if multiple outgroup species carry the same allele at a site, the simplest explanation is that this allele was present in the common ancestor, and the alternative allele in the focal species arose by mutation. However, several complications arise in practice:

1. **Incomplete lineage sorting (ILS):** When ancestral populations were large and speciation events were temporally close, gene trees may disagree with the species tree at a non-trivial

fraction of loci [Degnan and Rosenberg, 2009, Hobolth et al., 2011]. For the human–chimpanzee–gorilla clade, ILS affects approximately 1–3% of the genome [Scally et al., 2012].

2. **Back mutations and convergent substitutions:** The outgroup may have undergone an independent mutation at the same site, masking the true ancestral state.

3. **Alignment artifacts:** Segmental duplications, transposable elements, and structural variants can cause incorrect alignments that propagate into erroneous ancestral calls.

The Ensembl Enredo–Pecan–Ortheus (EPO) pipeline [Paten et al., 2008] addresses these challenges through multiple whole-genome alignment and phylogenetic reconstruction, but its computational demands are substantial, and extending it to new species requires significant infrastructure. Simpler approaches that use a single outgroup species are fast but susceptible to all three error modes described above.

Here we present **ancify** (version 1.3.0), an open-source Python pipeline that occupies a practical middle ground. It uses pairwise net-AXT alignments from the UCSC Genome Browser [Kent et al., 2003]—which are readily available for hundreds of species pairs—and infers ancestral alleles through one of three methods: two-tier outgroup voting, Fitch parsimony, or a machine learning classifier. The pipeline is species-agnostic, configuration-driven, and includes an optional GPU-accelerated backend that processes the entire human genome in under two minutes.

2 Methods

2.1 Pipeline overview

ancify operates in three sequential phases (Figure 1):

Phase 1 — Coordinate Projection
Pairwise net-AXT alignment files are parsed and each outgroup sequence is projected onto the focal species' coordinate system, producing one FASTA file per (outgroup, chromosome) pair.

Phase 2 — Ancestral Calling
Projected sequences are compared position-by-position using one of three inference methods (voting, parsimony, or ML) to determine the ancestral allele with case-encoded confidence.

Phase 3 — Evaluation (optional)
Ancestral calls are compared against an external reference (e.g., Ensembl EPO) and/or VCF variant sites to quantify accuracy and coverage.

All behavior is controlled by a single YAML configuration file that specifies the focal species, outgroup species with their alignment files, inference method, and optional evaluation parameters.

2.2 Phase 1: Coordinate projection

The pipeline reads pairwise *net* AXT alignment files from UCSC. These represent best-in-genome, one-to-one alignments between the focal and each outgroup species, produced by the netting algorithm that resolves overlapping alignment chains by retaining only the highest-scoring chain at each position [Kent et al., 2003].

Each AXT alignment block contains a header line with coordinates, followed by the focal (target) and outgroup (query) sequences. For each block, the projection algorithm maps outgroup bases onto the focal genome's coordinates:

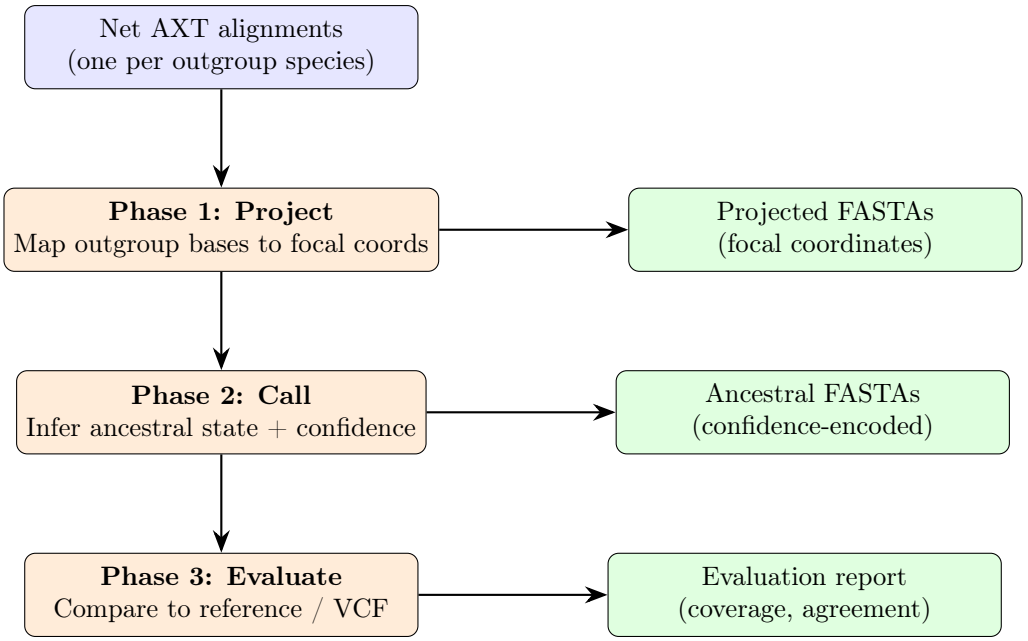

Figure 1: Overview of the ancify pipeline. Three sequential phases transform raw pairwise alignments into confidence-encoded ancestral FASTA files. Phase 3 is optional.

- If the focal position is a nucleotide, the corresponding outgroup base is recorded at that focal position.
- If the focal position is a gap character ($-$), the position is skipped (it represents an insertion in the outgroup with no corresponding focal coordinate).
- Positions not covered by any alignment block are filled with `N` (missing data).

The result is a FASTA file for each (outgroup, chromosome) pair with the same length as the focal chromosome. The implementation uses a two-pass architecture:

1. **Parse pass:** Stream through the AXT file, collecting block metadata (start position, target sequence, query sequence) for the target chromosome.
2. **Scatter pass:** Convert sequences to NumPy byte arrays and fill the output array using vectorized indexing operations.

The scatter operation replaces the original per-character Python loop with a vectorized NumPy operation. For each alignment block with focal sequence $\mathbf{h}$ and outgroup sequence $\mathbf{c}$ starting at position s:

$$\text{seq}[\text{pos}[j]] \leftarrow c_j \quad \forall\, j \text{ where } h_j \in \{A, C, G, T, N\} \tag{1}$$

where $\text{pos}[j] = s - 1 + |\{k \leq j : h_k \in \{A, C, G, T, N\}\}| - 1$ is the cumulative count of non-gap focal characters up to index j, mapped to the 0-based focal coordinate.

2.3 Phase 2: Ancestral state inference

Phase 2 determines the ancestral allele at every genomic position. Three inference methods are supported, sharing a common confidence-encoding scheme.

2.3.1 Method 1: Two-tier outgroup voting

The default method partitions outgroup species into two phylogenetic tiers. *Inner* outgroups are closely related species whose sequences cover a large fraction of the genome but may share derived alleles through ILS. *Outer* outgroups are more distantly related species that serve as an independent evolutionary check.

At each position i, the algorithm proceeds as follows:

```
Input:  Inner bases B_in, outer bases B_out,
        thresholds f_in, f_out
Output: Confidence-encoded ancestral character a_i

c_in  <- MajorityVote(B_in,  f_in)
c_out <- MajorityVote(B_out, f_out)

if c_in != N and c_in == c_out:
    return c_in              # High confidence (UPPERCASE)
elif c_in == N and c_out != N:
    return lower(c_out)      # Low confidence (lowercase)
elif c_in != N and c_out == N:
    return lower(c_in)       # Low confidence (lowercase)
elif c_in == N and c_out == N:
    return N                 # No data
else:
    return n                 # Disagreement (unresolved)
```

Listing 1: Two-tier outgroup voting (pseudocode).

The MajorityVote function selects the most frequent nucleotide among valid bases $\{A, C, G, T\}$, breaking ties alphabetically for determinism ($A > C > G > T$). If no allele reaches the minimum frequency threshold, the consensus is N (missing).

The two-tier design guards against ILS: because the inner outgroups are closely related, they are not independent observations—they may all share a derived allele inherited from a polymorphic ancestral population. The outer outgroup, being phylogenetically more distant, provides an independent check. When both tiers agree, the call is high-confidence; when they disagree, the position is flagged rather than erroneously resolved.

The `min_inner_freq` and `min_outer_freq` parameters control a coverage–accuracy trade-off. With k inner outgroups and `min_inner_freq`$= m$, at least m species must agree for a consensus. Higher thresholds increase specificity at the cost of marking more positions as missing.

2.3.2 Method 2: Fitch parsimony

The second method applies the Fitch algorithm [Fitch, 1971] to reconstruct the most parsimonious ancestral state at the root of a user-supplied phylogenetic tree. This method is preferred when the user has three or more outgroups at varying phylogenetic distances and wants the tree topology to determine how species are weighted.

The algorithm operates in two passes over the tree at each genomic position:

```
Input:  Rooted tree T, observed alleles {a_l} at leaves
Output: Root allele a_hat, ambiguity flag

--- Pass 1: Bottom-up (post-order) ---
for each leaf l:
    if a_l in {A, C, G, T}:
        S(l) <- {a_l}
    else:
        S(l) <- {A, C, G, T}     # Missing: wildcard
```

```
for each internal node v (post-order):
    if intersection of children's sets != empty:
        S(v) <- intersection of S(u) for u in children(v)
    else:
        S(v) <- union of S(u) for u in children(v)

--- Pass 2: Top-down (pre-order) ---
for each node v (pre-order, from root):
    if parent's allele in S(v):
        a_hat(v) <- parent's allele   # Minimize mutations
    else:
        a_hat(v) <- min(S(v))         # Alphabetical tie-break

a_hat    <- a_hat(root)
ambiguous <- |S(root)| > 1
```

Listing 2: Fitch parsimony for ancestral state reconstruction (pseudocode).

The intersection step identifies alleles compatible with all children (no mutation needed on those branches); the union step applies when at least one mutation is required. Missing data at a leaf is handled by assigning the full allele set $\{A, C, G, T\}$, making the leaf compatible with any state and ensuring that missing data never forces unnecessary mutations.

The Newick tree is parsed by a recursive-descent parser that supports branch lengths (parsed but unused by the Fitch algorithm) and both quoted and unquoted labels.

Comparison with voting. Consider a position where bonobo=G, chimp=G, gorilla=A, macaque=A. The voting method with inner= {bonobo, chimp, gorilla} yields inner consensus G (majority 2:1), outer consensus A, and flags the position as unresolved (n). Fitch parsimony on the tree `(((bonobo,chimp),gorilla),macaque)` infers root state A with a single G→A mutation on the bonobo–chimp branch—a high-confidence call.

2.3.3 Method 3: Machine learning classifier

The third method trains a LightGBM [Ke et al., 2017] gradient-boosted multi-class classifier (4 classes: A, C, G, T) to predict the ancestral allele from a per-position feature vector. This approach can learn substitution biases—such as elevated C→T rates at CpG sites—and local sequence context that the rule-based methods do not capture.

Feature engineering. For each genomic position, a feature vector of dimension $K + 9$ is constructed from K outgroup species (Table 1).

The first K features capture raw outgroup data. The next five features encode the same information the voting method uses, enabling the model to learn when voting is reliable. The final four context features capture position-dependent substitution biases.

Training. Two label sources are supported:

1. **Self-supervised:** Positions where the voting method's inner and outer tiers agree (uppercase output) serve as training labels. No external data is needed.
2. **Reference-supervised:** An external ancestral reference (e.g., Ensembl EPO) provides ground-truth labels.

Training subsamples up to ∼5 million sites per chromosome, stratified by allele class to maintain class balance. The LightGBM model uses 200 boosting iterations with 63 leaves per tree, learning rate 0.1, and 80% feature and bagging fractions.

Table 1: Per-position features for the ML classifier. K denotes the total number of outgroup species.

Feature	Description	Count
Outgroup bases	Integer-encoded base per outgroup (0–3; −1 for missing)	K
Data count	Number of outgroups with valid data	1
Agreement ratio	Fraction of valid outgroups agreeing with majority	1
Inner consensus	Majority base among inner outgroups (0–4)	1
Outer consensus	Majority base among outer outgroups (0–4)	1
Voting agreement	Whether inner = outer (binary)	1
GC content	GC fraction in ±50 bp window	1
CpG flag	Whether site is part of a CpG dinucleotide	1
Flanking bases	Bases immediately upstream and downstream	2

Table 2: Confidence encoding scheme shared by all inference methods. Typical fractions are for human autosomes with the BCGM outgroup configuration.

Character	Confidence	Condition	Typical fraction
`ACGT`	High	Both tiers agree / unambiguous parsimony / $p \geq 0.8$	∼75%
`acgt`	Low	One tier only / ambiguous parsimony / $0.5 \leq p < 0.8$	∼15%
`n`	Unresolved	Tiers disagree / $p < 0.5$	∼0.1%
`N`	Missing	No data from either tier / all outgroups missing	∼10%

Confidence calibration. The predicted class probability p maps to confidence levels:

$$a_i = \begin{cases} \text{uppercase}(c_i) & \text{if } p \geq \theta_{\text{high}} \\ \text{lowercase}(c_i) & \text{if } \theta_{\text{low}} \leq p < \theta_{\text{high}} \\ n & \text{if } p < \theta_{\text{low}} \\ N & \text{if all outgroups missing} \end{cases} \tag{2}$$

where c_i is the predicted base and $\theta_{\text{high}} = 0.8$, $\theta_{\text{low}} = 0.5$ by default. These thresholds can be tuned to trade precision for recall.

2.4 Confidence encoding

All three methods produce output in a unified FASTA format where letter case encodes confidence (Table 2). This convention is compatible with the Ensembl EPO ancestral sequence format, allowing seamless integration with existing workflows.

This encoding means that a single FASTA file carries both the ancestral call and its reliability, eliminating the need for sidecar quality files. Downstream analyses can select their stringency: conservative analyses use only uppercase positions, while maximum-coverage analyses include lowercase calls.

3 Implementation

3.1 Software architecture

ancify is implemented in Python (≥3.8) and organized as a modular package (Figure 2). Core dependencies are limited to PyYAML and NumPy; optional dependencies include PyTorch

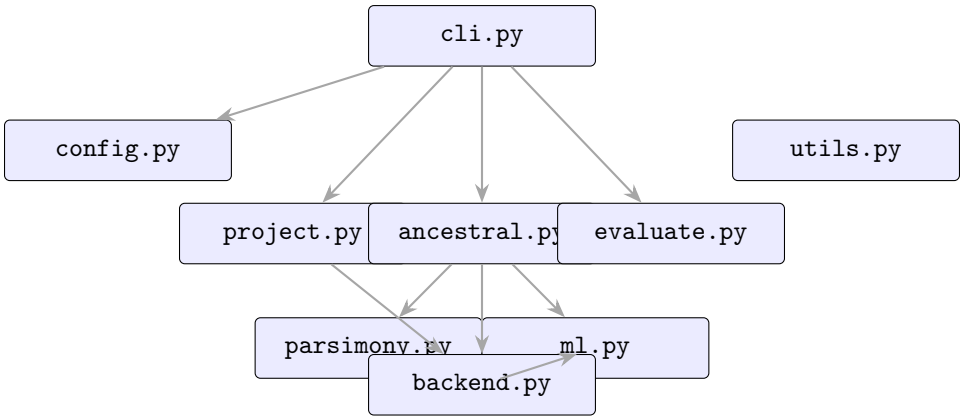

Figure 2: Module dependency graph of the ancify package. The `backend` module abstracts CPU (NumPy) and GPU (PyTorch) execution.

(GPU acceleration), LightGBM (ML method), `isal` (faster gzip I/O), and `scikit-allel` (VCF evaluation).

`cli.py`
Command-line interface with subcommands `init`, `project`, `call`, `evaluate`, `run`, and `train`.

`config.py`
YAML configuration loading, validation, and dataclass-based representation (`PipelineConfig`, `OutgroupSpec`, `EvaluationConfig`).

`project.py`
Phase 1: two-pass AXT parsing and vectorized scatter-based coordinate projection.

`ancestral.py`
Phase 2: dispatches to voting, parsimony, or ML calling with parallel execution across chromosomes.

`parsimony.py`
Fitch algorithm implementation: tree data structure (`TreeNode`), recursive-descent Newick parser, bottom-up and top-down traversals.

`ml.py`
Feature extraction, LightGBM training, prediction, and confidence encoding.

`evaluate.py`
Phase 3: coverage statistics, reference comparison, and VCF concordance.

`utils.py`
FASTA I/O (handles gzip transparently), chromosome lengths parsing, and the scalar majority vote function.

`backend.py`
CPU/GPU backend abstraction: auto-detection, sequence encoding, vectorized majority vote, vectorized ancestral calling, and vectorized block scatter.

3.2 GPU acceleration

The backend module provides transparent GPU acceleration through PyTorch tensor operations. Sequences are encoded as `uint8` arrays ($A = 0, C = 1, G = 2, T = 3$; all else= 4). The vectorized

Table 3: Wall-clock time for the full human genome (hg38, 4 outgroups) under different backends. GPU timings are for a single NVIDIA A100.

Component	Scalar Python	Vectorized CPU	GPU (PyTorch)
Phase 1 (projection)	∼4 hours	∼5 minutes	∼2 minutes
Phase 2 (calling)	∼8 hours	∼10 minutes	∼10 seconds
Total	∼12 hours	∼15 minutes	∼2 minutes

majority vote replaces per-position Python calls with column-wise tensor operations over the entire chromosome:

$$\text{counts}[b,:] = \sum_{s=1}^{K} \mathbb{1}\left[\text{encoded}[s,:] = b\right], \quad b \in \{0,1,2,3\} \tag{3}$$

$$\text{winner}[:] = \arg\max_{b} \text{counts}[b,:], \quad \text{winner}[j] = 4 \text{ if } \max_{b} \text{counts}[b,j] < f_{\min} \tag{4}$$

The confidence assignment uses approximately 15 tensor operations for an entire chromosome, replacing 248 million Python function calls for human chr1. The encoding of bases as consecutive integers 0–3 ensures that `torch.max(dim=0)` returns the first index among tied maxima, reproducing the alphabetical tie-breaking of the scalar implementation. All code paths produce bit-identical output, verified by the test suite.

Multi-GPU processing distributes chromosomes across available CUDA devices in round-robin fashion. Memory requirements are modest: approximately 6 GB per chromosome for human chr1 with four outgroups, fitting comfortably on consumer GPUs with 8–12 GB VRAM.

Table 3 summarizes the performance characteristics.

3.3 Configuration system

All pipeline behavior is controlled by a single YAML configuration file. A minimal configuration specifies the focal species label, a chromosome lengths file, the outgroup species with their alignment files, and an output directory. Listing 3 shows a typical configuration for human ancestral allele calling.

```yaml
focal_species: human
chromosome_lengths: chromoLens.txt

outgroups:
  inner:
    - name: bonobo
      alignment: hg38.panPan3.net.axt.gz
    - name: chimp
      alignment: hg38.panTro6.net.axt.gz
    - name: gorilla
      alignment: hg38.gorGor6.net.axt.gz
  outer:
    - name: macaque
      alignment: hg38.rheMac10.net.axt.gz

output_dir: ./ancestral_calls
num_cpus: 24
```

Listing 3: Minimal YAML configuration for human (hg38) ancestral calling with four outgroups.

Table 4: Example outgroup configurations for different focal species. Divergence times are approximate.

Focal species	Inner outgroups	Outer outgroup	Divergence
Human	Bonobo, Chimp, Gorilla	Macaque	6–25 Mya
Mouse	Rat	Rabbit	12–90 Mya
D. melanogaster	*D. simulans*, *D. sechellia*	*D. yakuba*	2.5–6 Mya
Brassica rapa	*B. oleracea*	*A. thaliana*	20 Mya
Zebrafish	Medaka	Fugu	—

Table 5: Validation of ancify BCGM calls against the Ensembl EPO 13-primate ancestral reference and 1000 Genomes Phase 3 VCFs. Coverage is defined as the fraction of positions with a non-`N` ancestral call.

Metric	chr1	chr22	chrX
Coverage (ancify BCGM)	79.1%	74.6%	44.4%
Coverage (Ensembl EPO)	90.2%	81.2%	44.1%
Disagreement rate	0.08%	0.11%	3.51%
Matches REF or ALT (VCF)	99.6%	99.5%	99.3%

Optional fields include `method` (`"voting"`, `"parsimony"`, or `"ml"`), `tree` (Newick string or file path), `backend` (`"auto"`, `"cpu"`, or `"gpu"`), `gpu_devices`, frequency thresholds, and an `evaluation` block for Phase 3.

3.4 Species agnosticism

ancify is not restricted to any particular organism. It operates on standard AXT alignment files that are available from the UCSC Genome Browser for hundreds of species pairs. Table 4 shows example configurations for several model and non-model organisms.

4 Validation

4.1 Comparison with Ensembl EPO

The BCGM (bonobo + chimp + gorilla + macaque) calling method was validated against the Ensembl EPO 13-primate ancestral reference for human GRCh38. Table 5 summarizes the results.

Autosomal chromosomes exhibit excellent agreement with the EPO reference, with disagreement rates below 0.12%. The higher disagreement on chrX (3.51%) reflects the reduced effective population size and male hemizygosity, which increase ILS rates in the human–great ape clade.

The coverage difference between ancify (79.1% on chr1) and EPO (90.2%) is expected: ancify uses 4 species whereas EPO uses 13, providing greater alignment coverage. Including low-confidence (lowercase) positions raises ancify's coverage from approximately 79% to approximately 90% on chr1.

4.2 VCF concordance

At known variant sites from the 1000 Genomes Project, the ancestral call matches either the REF or ALT allele in >99.5% of cases. The approximately 95% REF-matching fraction is expected, as reference genome assemblies are constructed from common alleles that are typically ancestral. The approximately 5% ALT-matching fraction identifies sites where the reference carries a derived allele—precisely the sites where polarization provides the most value.

The ∼0.5% of sites matching neither REF nor ALT are attributable to triallelic sites (where both REF and ALT are derived and the true ancestral allele is a third base), multi-allelic VCF records, and rare calling errors.

4.3 Built-in evaluation

Phase 3 computes three categories of metrics:

1. **Coverage statistics:** The fraction of positions at each confidence level (high, low, unresolved, missing).
2. **Reference comparison:** Position-by-position agreement and disagreement rates against an external ancestral reference.
3. **VCF comparison:** The fraction of variant sites where the ancestral call matches REF, ALT, or neither.

Results are written as structured text files in `<output_dir>/evaluation/`, one per chromosome.

5 Usage

5.1 Installation

```
pip install .                    # core (CPU)
pip install '.[evaluate]'        # + evaluation dependencies
pip install '.[fast]'            # + faster gzip (isal)
pip install '.[ml]'              # + LightGBM, scikit-learn

# GPU acceleration (PyTorch with CUDA)
pip install torch --index-url https://download.pytorch.org/whl/cu128
```

5.2 Command-line interface

The pipeline is invoked through the `ancify` command:

```
ancify init      [-o FILE]            # Generate template config
ancify project   -c CONFIG [-n N]     # Phase 1: project alignments
ancify call      -c CONFIG [-n N]     # Phase 2: call ancestral states
ancify evaluate  -c CONFIG [-n N]     # Phase 3: evaluate (optional)
ancify run       -c CONFIG [-n N]     # All phases end-to-end
ancify train     -c CONFIG [-o MODEL]# Train ML model
```

Alternatively: `python -m ancify run -c config.yaml`.

5.3 Python API

For programmatic use, all pipeline functions are importable:

```
from ancify.config import load_config
from ancify.project import run_projection
from ancify.ancestral import run_ancestral_calling
from ancify.ancestral import call_ancestral_base

# Full pipeline
cfg = load_config("config.yaml")
```

```python
run_projection(cfg)
run_ancestral_calling(cfg)

# Single-position call
base = call_ancestral_base(
    inner_bases=["A", "A", "G"],
    outer_bases=["A"],
)  # Returns "A" (high confidence)
```

Listing 4: Programmatic usage of the ancify Python API.

5.4 Downstream integration

The output ancestral FASTA files integrate directly into standard population genetics workflows:

```python
from ancify.utils import read_fasta

_, seq = read_fasta("ancestral_calls/chr1.fa")

for variant in vcf:
    anc = seq[variant.POS - 1].upper()
    if anc == variant.REF:
        pass  # REF is ancestral
    elif anc == variant.ALT:
        pass  # ALT is ancestral (flip frequencies)
```

Listing 5: Polarizing a VCF using ancify output.

6 Discussion

6.1 Design philosophy

ancify was designed with three priorities. First, *simplicity*: a single YAML file controls all behavior, and the pipeline reads standard file formats available for hundreds of species. Second, *transparency*: the case-encoding of confidence levels enables users to make informed decisions about which positions to trust without requiring auxiliary files or probabilistic models. Third, *performance*: the GPU backend makes genome-wide analysis feasible in minutes rather than hours, enabling rapid iteration during method development or when processing multiple species.

6.2 Comparison of inference methods

The three methods offer complementary strengths. The two-tier voting method is the simplest, requiring no phylogenetic tree and making minimal assumptions beyond the inner/outer partition. It is well-suited for the common case of 3–4 outgroups with a clear phylogenetic structure.

Fitch parsimony is more principled when the outgroup phylogeny is known, as it accounts for the tree topology rather than treating species as exchangeable within tiers. It resolves many positions that voting marks as unresolved—particularly ILS-affected sites where the tree structure disambiguates the ancestral state.

The ML classifier is most powerful when training data is available (either self-supervised from high-confidence voting sites or reference-supervised from an external ancestral sequence). It can learn substitution biases, CpG effects, and local sequence context, and provides calibrated probability scores rather than binary confidence levels. However, it requires an additional training step and may overfit to biases present in the training data.

6.3 Limitations

Several limitations should be noted:

1. **Alignment dependence.** The accuracy of all methods is bounded by the quality of the input alignments. Regions with segmental duplications, recent transposable element insertions, or structural variants may produce incorrect projections.

2. **No substitution model.** The voting and parsimony methods do not account for branch-length-weighted substitution probabilities. More sophisticated probabilistic methods (e.g., maximum likelihood on a substitution model) could improve accuracy at the cost of additional complexity.

3. **Coverage limited by outgroup count.** With only 4 species, ancify's coverage (79% high-confidence on human chr1) is lower than EPO's (90%), which uses 13 species. Adding more outgroups would improve coverage.

4. **ILS on the X chromosome.** The elevated disagreement rate on chrX (3.51% vs. <0.12% on autosomes) reflects genuine biological challenges that would require more outgroups or probabilistic modeling to address.

6.4 Future directions

Planned extensions include support for probabilistic ancestral reconstruction using continuous-time Markov models on the phylogenetic tree, integration with `hal` format whole-genome alignments, and automated outgroup selection based on alignment coverage and phylogenetic distance.

7 Availability

ancify is released under the MIT license. Source code, documentation, and example configurations are available at:

`https://github.com/kevinkorfmann/ancify`

Full documentation including tutorials, algorithm deep dives, and a species adaptation guide is hosted at `https://ancify.readthedocs.io`.

Acknowledgments

The UCSC Genome Browser group is acknowledged for providing the pairwise alignment infrastructure that makes this pipeline possible. The Ensembl project is acknowledged for the EPO ancestral reference used in validation.

References

Michael DeGiorgio, Christian D. Huber, Melissa J. Hubisz, Ines Hellmann, and Rasmus Nielsen. SweepFinder2: increased sensitivity, robustness and flexibility. *Bioinformatics*, 32(12):1895–1897, 2016. doi: 10.1093/bioinformatics/btw051.

James H. Degnan and Noah A. Rosenberg. Gene tree discordance, phylogenetic inference and the multispecies coalescent. *Trends in Ecology & Evolution*, 24(6):332–340, 2009. doi: 10.1016/j.tree.2009.01.009.

Justin C. Fay and Chung-I Wu. Hitchhiking under positive Darwinian selection. *Genetics*, 155 (3):1405–1413, 2000. doi: 10.1093/genetics/155.3.1405.

Walter M. Fitch. Toward defining the course of evolution: minimum change for a specific tree topology. *Systematic Biology*, 20(4):406–416, 1971. doi: 10.1093/sysbio/20.4.406.

Ryan N. Gutenkunst, Ryan D. Hernandez, Scott H. Williamson, and Carlos D. Bustamante. Inferring the joint demographic history of multiple populations from multidimensional SNP frequency data. *PLoS Genetics*, 5(10):e1000695, 2009. doi: 10.1371/journal.pgen.1000695.

Asger Hobolth, Julien Y. Dutheil, John Hawks, Mikkel H. Schierup, and Thomas Mailund. Incomplete lineage sorting patterns among human, chimpanzee, and orangutan suggest recent orangutan speciation and widespread selection. *Genome Research*, 21(3):349–356, 2011. doi: 10.1101/gr.114751.110.

Julien Jouganous, Will Long, Aaron P. Ragsdale, and Simon Gravel. Inferring the joint demographic history of multiple populations: beyond the diffusion approximation. *Genetics*, 206(3): 1549–1567, 2017. doi: 10.1534/genetics.117.200493.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30:3146–3154, 2017.

W. James Kent, Robert Baertsch, Angie Hinrichs, Webb Miller, and David Haussler. Evolution's cauldron: duplication, deletion, and rearrangement in the mouse and human genomes. *Proceedings of the National Academy of Sciences*, 100(20):11484–11489, 2003. doi: 10.1073/pnas.1932072100.

Motoo Kimura. The neutral theory of molecular evolution. *Cambridge University Press*, 1983.

Rasmus Nielsen, Scott Williamson, Yuseob Kim, Melissa J. Hubisz, Andrew G. Clark, and Carlos Bustamante. Genomic scans for selective sweeps using SNP data. *Genome Research*, 15(11): 1566–1575, 2005. doi: 10.1101/gr.4252305.

Benedict Paten, Javier Herrero, Stephen Fitzgerald, Kathryn Beal, Paul Flicek, Ian Holmes, and Ewan Birney. Genome-wide nucleotide-level mammalian ancestor reconstruction. *Genome Research*, 18(11):1829–1843, 2008. doi: 10.1101/gr.076521.108.

Aylwyn Scally, Julien Y. Dutheil, LaDeana W. Hillier, Gregory E. Jordan, Ian Goodhead, Javier Herrero, Asger Hobolth, Tuuli Lappalainen, Thomas Mailund, Tomas Marques-Bonet, et al. Insights into hominid evolution from the gorilla genome sequence. *Nature*, 483(7388):169–175, 2012. doi: 10.1038/nature10842.

G. A. Watterson. On the number of segregating sites in genetical models without recombination. *Theoretical Population Biology*, 7(2):256–276, 1975. doi: 10.1016/0040-5809(75)90020-9.